

【赛道二】 【沐曦】 Clike优化赛道

1. 参赛者需要提前了解的内容

1.1 智能算子迁移平台：KernelSwift

KernelSwift 是由上海人工智能实验室 DeepLink 与书生提供技术支持的**智能算子迁移平台**，支持将 PyTorch 算子代码自动适配到不同硬件（如英伟达、华为昇腾、海光、寒武纪等），并通过自动调优生成高性能算子实现，同时提供算子广场复用已优化算子。

详细操作手册可看文档：[📖 KernelSwift 智能算子生成系统操作手册](#)。

参赛者可以借助KernelSwift平台能力，执行算子迁移定制化服务。并且在平台生成算子的基础上，可手动优化算子表现，最终获得参赛者最理想的结果并上传评分。

参赛对象

个人参赛或者团队参赛（团队人数 2—5人）

报名方式

https://deeplink.org.cn/kernelswift_competition

1.2 算力资源申请

请先前往报名官网，找到算力领取页面，并领取所需算力。

原则上每个厂商可领取一次资源，有三种情况可联系大赛小助手额外领取：

- （1）报名页面可领取范围之外的厂商算力的情况
- （2）报名团队赛的组员，需要单独领取的情况
- （3）已领取算力使用完毕，需要补充算力的情况

1.3 赛道指定平台&芯片

平台（排名不分先后）	芯片
沐曦	C500

2. 奖项设置

2.1 奖励规则

待后续公布

2.2 奖项设置

赛道	赛题数	奖项	数量	奖金（元）
赛道二（沐曦）	3	一等奖	1	6000
		二等奖	1	3000
		三等奖	2	1000

说明：赛题单独排名、分开发奖。以上奖金均为税前奖金，仅作参考，奖金可能随奖金池变化而变化，以最终发布结果为准

3. 赛题说明

Task01: engram_hash

代码块

```
1  import torch
2  import torch.nn as nn
3  class Model(nn.Module):
4      def __init__(self):
5          super(Model, self).__init__()
6      def forward(
7          self,
8          ngram_token_ids: torch.Tensor,
9          multipliers: torch.Tensor,
10         vocab_sizes: torch.Tensor,
11         offsets: torch.Tensor,
12     ) -> torch.Tensor:
13         """Pure PyTorch reference implementation of engram hash.
14         Args:
15             ngram_token_ids: N-gram token IDs of shape (num_tokens,
max_ngram_size), int32.
16             multipliers: Per-layer hash multipliers of shape
(num_ngram_layers, max_ngram_size), int64.
17             vocab_sizes: Per-layer per-ngram embedding table sizes of shape
(num_ngram_layers, max_ngram_size - 1,
num_embed_table_per_ngram), int32.
18             offsets: Per-layer embedding table offsets of shape
```

```

20         (num_ngram_layers, (max_ngram_size - 1) *
num_embed_table_per_ngram), int32.
21     Returns:
22         Embedding indices of shape (num_ngram_layers, num_tokens,
(max_ngram_size - 1) * num_embed_table_per_ngram), int32.
23     """
24     num_ngram_layers = multipliers.shape[0]
25     max_ngram_size = multipliers.shape[1]
26     prod = ngram_token_ids.to(torch.int64).unsqueeze(0) *
multipliers.unsqueeze(1)
27     ans = [[] for _ in range(num_ngram_layers)]
28     hashes = prod[:, :, 0].clone()
29     for i in range(1, max_ngram_size):
30         hashes.bitwise_xor_(prod[:, :, i])
31         for layer_idx in range(num_ngram_layers):
32             ans[layer_idx].append((hashes[layer_idx].unsqueeze(-1) %
vocab_sizes[layer_idx, i - 1].to(torch.int64).unsqueeze(0)).to(torch.int32))
33     for layer_idx in range(num_ngram_layers):
34         ans[layer_idx] = torch.cat(ans[layer_idx], dim=-1)
35     output = torch.stack(ans, dim=0)
36     return output + offsets.unsqueeze(1)
37 def make_offsets(vocab_sizes: torch.Tensor) -> torch.Tensor:
38     """Compute exclusive prefix-sum offsets from vocab_sizes.
39     Args:
40         vocab_sizes: Per-layer per-ngram embedding table sizes of shape
41         (num_ngram_layers, max_ngram_size - 1, num_embed_table_per_ngram),
int32.
42     Returns:
43         Offsets of shape (num_ngram_layers, (max_ngram_size - 1) *
num_embed_table_per_ngram), int32.
44     """
45     num_ngram_layers = vocab_sizes.shape[0]
46     offsets_list = []
47     for layer_idx in range(num_ngram_layers):
48         flat = vocab_sizes[layer_idx].view(-1)
49         prefix = torch.cat([torch.zeros(1, dtype=torch.int32,
device=flat.device), flat[:-1].cumsum(0, dtype=torch.int32)])
50         offsets_list.append(prefix)
51     return torch.stack(offsets_list, dim=0)
52 def generate_test_data(params):
53     num_tokens = params['num_tokens']
54     max_ngram_size = params['ngram']
55     num_ngram_layers = params['layers']
56     num_embed_table_per_ngram = params['tables']
57     ngram_token_ids = torch.randint(0, 100000, (num_tokens, max_ngram_size),
dtype=torch.int32)

```

```

58     multipliers = torch.randint(0, 100000, (num_ngram_layers, max_ngram_size),
dtype=torch.int64)
59     vocab_sizes = torch.randint(100000, 1000000, (num_ngram_layers,
max_ngram_size - 1, num_embed_table_per_ngram), dtype=torch.int32)
60     offsets = make_offsets(vocab_sizes)
61     return (ngram_token_ids, multipliers, vocab_sizes, offsets)
62 def test_engram_hash():
63     return Model(*get_init_inputs()).forward(*get_inputs())
64 def get_inputs():
65     params = {'num_tokens': 4096}
66     num_tokens = params['num_tokens']
67     max_ngram_size = 3
68     num_ngram_layers = 2
69     num_embed_table_per_ngram = 8
70     ngram_token_ids, multipliers, vocab_sizes, offsets = generate_test_data(
71         {'num_tokens': num_tokens, 'ngram': max_ngram_size, 'layers':
num_ngram_layers, 'tables': num_embed_table_per_ngram})
72     return [ngram_token_ids, multipliers, vocab_sizes, offsets]
73 def get_init_inputs():
74     return []

```

Task02: Indexer

代码块

```

1  import torch
2  from torch import nn
3  import torch.nn.functional as F
4  from dataclasses import dataclass
5  from typing import Tuple, Optional, Literal
6  from functools import lru_cache
7  from contextlib import contextmanager
8  import math
9
10
11 world_size = 1
12 rank = 0
13 block_size = 128
14 fp4_block_size = 32
15 default_dtype = torch.bfloat16
16
17
18 @contextmanager
19 def set_dtype(dtype):
20     """Temporarily override torch default dtype, restoring it on exit (even if
an exception occurs)."""

```

```

21     prev = torch.get_default_dtype()
22     torch.set_default_dtype(dtype)
23     try:
24         yield
25     finally:
26         torch.set_default_dtype(prev)
27
28 @dataclass
29 class ModelArgs:
30     """Model hyperparameters. Field names match the config JSON keys."""
31     max_batch_size: int = 4
32     max_seq_len: int = 4096
33     dtype: Literal["bf16", "fp8"] = "fp8"
34     scale_fmt: Literal[None, "ue8m0"] = "ue8m0"
35     expert_dtype: Literal[None, "fp4"] = None
36     scale_dtype: Literal["fp32", "fp8"] = "fp8"
37     vocab_size: int = 129280
38     dim: int = 4096
39     moe_inter_dim: int = 4096
40     n_layers: int = 7
41     n_hash_layers: int = 0
42     n_mtp_layers: int = 1
43     n_heads: int = 64
44     # moe
45     n_routed_experts: int = 8
46     n_shared_experts: int = 1
47     n_activated_experts: int = 2
48     score_func: Literal["softmax", "sigmoid", "sqrtsoftplus"] = "sqrtsoftplus"
49     route_scale: float = 1.
50     swiglu_limit: float = 0.
51     # mqa
52     q_lora_rank: int = 1024
53     head_dim: int = 512
54     rope_head_dim: int = 64
55     norm_eps: float = 1e-6
56     o_groups: int = 8
57     o_lora_rank: int = 1024
58     window_size: int = 128
59     compress_ratios: Tuple[int] = (0, 0, 4, 128, 4, 128, 4, 0)
60     # yarn
61     compress_rope_theta: float = 40000.0
62     original_seq_len: int = 0
63     rope_theta: float = 10000.0
64     rope_factor: float = 40
65     beta_fast: int = 32
66     beta_slow: int = 1
67     # index

```

```

68     index_n_heads: int = 64
69     index_head_dim: int = 128
70     index_topk: int = 512
71     # hc
72     hc_mult: int = 4
73     hc_sinkhorn_iters: int = 20
74     hc_eps: float = 1e-6
75
76     class Linear(nn.Module):
77         """Linear layer supporting BF16, FP8, and FP4 weight formats with per-
78         block scaling."""
79
80         def __init__(self, in_features: int, out_features: int, bias: bool =
81         False, dtype = None):
82             super().__init__()
83             self.in_features = in_features
84             self.out_features = out_features
85             dtype = dtype or default_dtype
86             self.weight = nn.Parameter(torch.empty(out_features, in_features,
87             dtype=dtype))
88             nn.init.kaiming_uniform_(
89                 self.weight,
90                 a=math.sqrt(5),
91             )
92             self.register_parameter("scale", None)
93             if bias:
94                 self.bias = nn.Parameter(torch.empty(out_features))
95                 bound = 1 / math.sqrt(in_features)
96                 nn.init.uniform_(
97                     self.bias,
98                     -bound,
99                     bound,
100                 )
101             else:
102                 self.register_parameter("bias", None)
103
104         def forward(self, x: torch.Tensor) -> torch.Tensor:
105             return linear(x, self.weight, self.bias)
106
107     def linear(x: torch.Tensor, weight: torch.Tensor, bias: Optional[torch.Tensor]
108     = None) -> torch.Tensor:
109         """Dispatches to fp4_gemm / fp8_gemm / F.linear based on weight dtype.
110         For quantized weights, x is first quantized to FP8 via act_quant."""
111         assert bias is None
112         return F.linear(x, weight)
113
114     class ColumnParallelLinear(Linear):

```

```

111     """Shards output dim across TP ranks. No all-reduce needed on output."""
112     def __init__(self, in_features: int, out_features: int, bias: bool =
False, dtype = None):
113         assert out_features % world_size == 0, f"Output features must be
divisible by world size (world_size={world_size})"
114         self.part_out_features = out_features // world_size
115         super().__init__(in_features, self.part_out_features, bias, dtype)
116
117     def forward(self, x: torch.Tensor) -> torch.Tensor:
118         return linear(x, self.weight, self.bias)
119
120
121 class RowParallelLinear(Linear):
122     """Shards input dim across TP ranks. All-reduce on output to sum partial
results."""
123     def __init__(self, in_features: int, out_features: int, bias: bool =
False, dtype = None):
124         assert in_features % world_size == 0, f"Input features must be
divisible by world size (world_size={world_size})"
125         self.part_in_features = in_features // world_size
126         super().__init__(self.part_in_features, out_features, bias, dtype)
127
128     def forward(self, x: torch.Tensor) -> torch.Tensor:
129         y = linear(x, self.weight, None)
130         if self.bias is not None:
131             y += self.bias
132         return y.type_as(x)
133
134     def apply_rotary_emb(x: torch.Tensor, freqs_cis: torch.Tensor, inverse: bool =
False) -> torch.Tensor:
135         """Applies rotary positional embeddings in-place. Uses conjugate for
inverse (de-rotation)."""
136         y = x
137         x = torch.view_as_complex(x.float().unflatten(-1, (-1, 2)))
138         if inverse:
139             freqs_cis = freqs_cis.conj()
140         if x.ndim == 3:
141             freqs_cis = freqs_cis.view(1, x.size(1), x.size(-1))
142         else:
143             freqs_cis = freqs_cis.view(1, x.size(1), 1, x.size(-1))
144         x = torch.view_as_real(x * freqs_cis).flatten(-2)
145         y.copy_(x)
146         return y
147
148
149 class Model(torch.nn.Module):

```

```

150     """Selects top-k compressed KV positions for sparse attention via learned
151     scoring.
152     Has its own Compressor (with Hadamard rotation) to build compressed KV for
153     scoring."""
154
155     def __init__(self, args: ModelArgs, freqs_cis: torch.Tensor, kv_cache:
156     torch.Tensor, compress_ratio: int = 4):
157         super().__init__()
158         self.dim = args.dim
159         self.n_heads = args.index_n_heads
160         self.n_local_heads = args.index_n_heads // world_size
161         self.head_dim = args.index_head_dim
162         self.rope_head_dim = args.rope_head_dim
163         self.index_topk = args.index_topk
164         self.q_lora_rank = args.q_lora_rank
165         self.wq_b = ColumnParallelLinear(self.q_lora_rank, self.n_heads *
166     self.head_dim)
167         self.weights_proj = ColumnParallelLinear(self.dim, self.n_heads,
168     dtype=torch.bfloat16)
169         self.softmax_scale = self.head_dim ** -0.5
170         self.compress_ratio = compress_ratio
171         self.kv_cache = kv_cache
172         self.freqs_cis = freqs_cis
173
174     def forward(self, x: torch.Tensor, qr: torch.Tensor, start_pos: int,
175     offset: int):
176         bsz, seqlen, _ = x.size()
177         freqs_cis = self.freqs_cis[start_pos:start_pos+seqlen]
178         ratio = self.compress_ratio
179         rd = self.rope_head_dim
180         end_pos = start_pos + seqlen
181         q = self.wq_b(qr)
182         q = q.unflatten(-1, (self.n_local_heads, self.head_dim))
183         apply_rotary_emb(q[..., -rd:], freqs_cis)
184         weights = self.weights_proj(x) * (self.softmax_scale * self.n_heads **
185     -0.5)
186         index_score = torch.einsum("bshd,btd->bsht", q, self.kv_cache[:bsz,
187     :end_pos // ratio])
188         index_score = (index_score.relu_() * weights.unsqueeze(-1)).sum(dim=2)
189         if start_pos == 0:
190             mask = torch.arange(seqlen // ratio).repeat(seqlen, 1) >=
191     torch.arange(1, seqlen + 1).unsqueeze(1) // ratio
192             index_score += torch.where(mask, float("-inf"), 0)
193         topk_idx = index_score.topk(min(self.index_topk, end_pos // ratio),
194     dim=-1)[1]
195         if start_pos == 0:

```

```

186         mask = topk_idx >= torch.arange(1, seq_len + 1).unsqueeze(1) //
ratio
187         topk_idx = torch.where(mask, -1, topk_idx + offset)
188     else:
189         topk_idx += offset
190     return topk_idx
191
192
193 args = ModelArgs(
194     max_batch_size=8,
195     max_seq_len=2600,
196     dim=1024,
197     index_n_heads=16,
198     index_head_dim=64,
199     index_topk=128,
200     q_lora_rank=256,
201     rope_head_dim=32
202 )
203
204 def get_inputs():
205     batch_size = 8
206     seq_len = 2600
207     x = torch.randn(batch_size, seq_len, args.dim, dtype=torch.bfloat16)
208     qr = torch.randn(batch_size, seq_len, args.q_lora_rank,
dtype=torch.bfloat16)
209     start_pos = 0
210     offset = 0
211     return [x, qr, start_pos, offset]
212
213
214 def get_init_inputs():
215     compress_ratio=4
216     max_seq_len = args.max_seq_len
217     rope_theta = 10000.0
218     # freqs_cis = torch.zeros(max_seq_len, args.rope_head_dim)
219     freqs = 1.0 / (rope_theta ** (torch.arange(0, args.rope_head_dim, 2)
[:args.rope_head_dim//2].float() / args.rope_head_dim))
220     t = torch.arange(max_seq_len, dtype=torch.float32)
221     freqs = torch.outer(t, freqs).float()
222     freqs_cis = torch.polar(torch.ones_like(freqs), freqs).view(max_seq_len,
-1)
223     kv_cache = torch.randn(args.max_batch_size, args.max_seq_len //
compress_ratio, args.index_head_dim, dtype=default_dtype)
224     return [args, freqs_cis, kv_cache, compress_ratio]
225
226
227 if __name__ == "__main__":

```

```

228     result = Model(*get_init_inputs()).forward(*get_inputs())
229     print(f"Forward pass successful! Output shape: {result.shape}, dtype:
{result.dtype}")
230     print(f"Sample output values: {result[0, :5]}")

```

Task03: norm_fn

代码块

```

1  import torch
2  import torch.nn as nn
3
4  class Model(nn.Module):
5
6      def __init__(self):
7          super(Model, self).__init__()
8
9      def forward(
10         self,
11         residual: torch.Tensor,
12         mhc_fn: torch.Tensor,
13         mhc_norm_weight: torch.Tensor | None,
14         mhc_norm_eps: float,
15     ) -> torch.Tensor:
16         if mhc_norm_weight is not None:
17             mhc_fn = mhc_fn * mhc_norm_weight
18             residual = residual.flatten(2, 3).float()
19             assert mhc_fn.dtype == residual.dtype == torch.float
20             mhc_mult = mhc_fn.shape[0]
21             rms_group_size = mhc_fn.shape[-1]
22             mixes = torch.einsum(
23                 'mbk,nbk->mbn',
24                 residual.view(-1, 1, rms_group_size),
25                 mhc_fn.view(mhc_mult, 1, rms_group_size),
26             )
27             sqrsum = residual.view(-1, 1, rms_group_size).square().sum(-1)
28             mixes = (mixes * (sqrsum.unsqueeze(-1) / rms_group_size +
mhc_norm_eps).rsqrt()).sum(-2)
29             return mixes.view(*residual.shape[:2], -1)
30
31     n1=13
32     mhc_mult=4
33     hidden_size=1280
34     generate_normmw=False
35
36     def generate_norm_fn_test_data(

```

```

37     n1: int,
38     mhc_mult: int,
39     hidden_size: int,
40     generate_normw: bool,
41 ) -> dict[str, torch.Tensor]:
42     n0 = 1
43     mhc_mult3 = mhc_mult * (2 + mhc_mult)
44     mhc_hidden_size = mhc_mult * hidden_size
45
46     residual = (
47         torch.randn((n0, n1, mhc_mult, hidden_size), dtype=torch.float)
48         .mul(1 + torch.arange(mhc_mult).mul(0.01).view(1, 1, -1, 1))
49         .bfloat16()
50     )
51
52     fn = (
53         torch.randn((mhc_mult3, mhc_mult, hidden_size), dtype=torch.float)
54         * 1e-4
55         * (1 + torch.arange(mhc_mult).mul(0.01).view(1, -1, 1))
56     ).flatten(1, 2)
57
58     if generate_normw:
59         normw = torch.randn((mhc_hidden_size,), dtype=torch.float) * 0.1 + 1.0
60     else:
61         normw = None
62
63     out_grad = torch.randn((n0, n1, mhc_mult3), dtype=torch.float)
64
65     return [residual, fn, normw, out_grad, 1e-6]
66
67
68 def get_inputs():
69     residual, fn, normw, out_grad, mhc_norm_eps =
70     generate_norm_fn_test_data(n1, mhc_mult, hidden_size, generate_normw)
71     return [residual, fn, None, mhc_norm_eps]
72
73 def get_init_inputs():
74     return []

```

4. 作品提交

4.1 提交方式

(1) 选手完成赛题后，将结果发送至DeepLink专属邮箱，最终形式以邮箱提交为准，可以提交代码文件或PR链接，PR提交规范参考如下4.3

deeplink@pjlab.org.cn

(2) 邮件名称格式：

【2026KernelSwift算子创新大赛】-参赛选手名称/团队名称-所选赛道-UID

(3) 参赛结果打包，压缩文件名称：

【2026KernelSwift算子创新大赛】-参赛选手名称/团队名称-所选赛题-UID，并作为附件提交。

*若提交方式发生更改，将会另行通知。

4.2 提交内容

提交作品需包含：

- 算子优化代码；
- README 文档；
- 环境配置文件；
- 运行脚本；
- 性能测试结果。

 评测代码详见Github: https://github.com/DeepLink-org/DLBlas/blob/main/benchmarks/ks/auto_bench.py#L35

4.3 PR规范

PR标题格式：

代码块

- 1 [KernelSwift算子优化]
- 2 参赛选手名称/团队名称-所选赛题-UID

PR内容需包含：

- 作品说明；
- 优化方案；
- 性能测试结果；
- 原创声明。

5. 评分规则

5.1 评分标准

- **正确性测试**：参赛代码须通过正确性校验，否则不进行排名。
- **加速比测试**：以 Speedup（加速比）为核心性能指标，按照加速比进行排名。加速比的基准是赛中给出的 reference 实现（torch实现）。
- **反对作弊**：反对采用异常捕获、条件分支等手段规避自定义算子执行、fallback至Pytorch内置算子。若代码实际执行路径未运行自定义算子，全程仅使用 PyTorch 内置算子，不计成绩，不参与排名。
- **特殊情况**：加速比相同者，按照提交时间排名。

5.2 代码要求

1. 你提交的代码必须包含与参考实现一致的python Model定义，包括__init__函数和forward函数的参数。在forward函数内部调用你的代码，必须通过python调用你实现的算子，否则系统无法评测。
2. 文件编码必须为 UTF-8
3. 推荐使用 Python 3.10

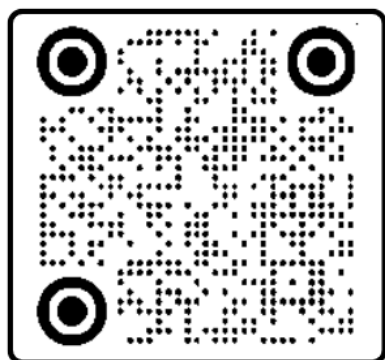
关于我们

💡想了解更多：

- 官方网站：[DeepLink官网](#)
- GitHub主页：[DeepLink开源社区](#)

💡我们也已在知乎、小红书台、CSDN等平台开通官方账号

欢迎大家关注DeepLink！获取最新产品动态、前沿技术分享及行业资讯！



小红书账号



知乎账号



CSDN账号